
Documentação Técnica do Projeto de Banco de Dados

Introdução ao Projeto

Descrição do Projeto

Este projeto foi desenvolvido para gerenciar um sistema de agendamento de serviços de TI, permitindo que os usuários possam gerenciar e analisar o trabalho.

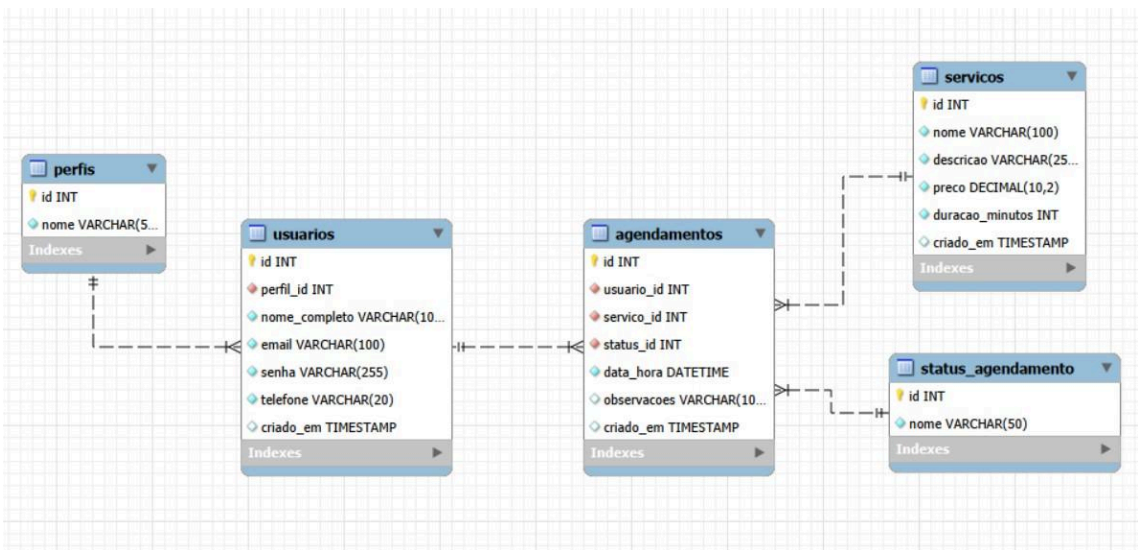
Objetivo do Banco de Dados

O objetivo do banco de dados é organizar e gerenciar informações sobre os agendamentos e clientes, bem como as horas gastas, garantindo a eficiência, garantindo eficiência no processamento de agendamentos e relatórios.

Modelagem de Dados

Diagrama Entidade-Relacionamento (DER)

O Diagrama Entidade-Relacionamento (DER) ilustra as entidades e relacionamentos do sistema, como mostrado abaixo:



Estrutura do Banco de Dados

Scripts de Criação do Banco de Dados

Os scripts SQL para criação das tabelas e seus relacionamentos são apresentados abaixo:

```
CREATE TABLE perfis (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    perfil_id INT NOT NULL DEFAULT 1,  
    nome_completo VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    senha VARCHAR(255) NOT NULL,  
    telefone VARCHAR(20) NOT NULL,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_usuario_perfil FOREIGN KEY (perfil_id) REFERENCES perfis(id)  
);
```

```
CREATE TABLE servicos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    descricao VARCHAR(250) NOT NULL,  
    preco DECIMAL(10, 2) NOT NULL DEFAULT 0.00 CHECK (preco >= 0),  
    duracao_minutos INT NOT NULL DEFAULT 60,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE status_agendamento (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE agendamentos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    usuario_id INT NOT NULL,  
    servico_id INT NOT NULL,  
    status_id INT NOT NULL DEFAULT 1,  
    data_hora DATETIME NOT NULL,  
    observacoes VARCHAR(100) NULL,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_agendamento_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,  
    CONSTRAINT fk_agendamento_servico FOREIGN KEY (servico_id) REFERENCES servicos(id) ON DELETE RESTRICT,  
    CONSTRAINT fk_agendamento_status FOREIGN KEY (status_id) REFERENCES status_agendamento(id) ON DELETE RESTRICT  
);
```

```
CREATE TABLE log_agendamentos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    agendamento_id INT NOT NULL,  
    status_anterior INT,  
    status_novo INT,  
    alterado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    alterado_por VARCHAR(100)  
);
```

Descrição das Tabelas

- **Tabela perfis:** Armazena os perfis de acesso ou tipos de usuários do sistema.
 - id: Identificador único do perfil.
 - nome: Nome do perfil (ex: "Administrador", "Cliente").
- **Tabela usuarios:** Armazena informações dos usuários cadastrados no sistema.
 - id: Identificador único do usuário.
 - perfil_id: Perfil associado ao usuário (padrão é 1).
 - nome_completo: Nome completo do usuário.
 - email: Endereço de e-mail único do usuário.
 - senha: Senha criptografada do usuário.
 - telefone: Contato telefônico.
 - criado_em: Data e hora de cadastro no sistema.
- **Tabela servicos:** Armazena informações sobre os serviços disponíveis para agendamento.
 - id: Identificador único do serviço.
 - nome: Nome do serviço prestado.
 - descricao: Descrição detalhada do serviço.
 - preco: Preço do serviço (não pode ser negativo).
 - duracao_minutos: Tempo estimado de duração do serviço em minutos.
 - criado_em: Data e hora de cadastro do serviço.
- **Tabela status_agendamento:** Armazena os estados possíveis em que um agendamento pode estar.
 - id: Identificador único do status.
 - nome: Nome do status (ex: "Pendente", "Confirmado", "Cancelado").

- **Tabela agendamentos:** Registra os agendamentos de serviços realizados pelos usuários.
 - id: Identificador único do agendamento.
 - usuario_id: Usuário que realizou o agendamento.
 - servico_id: Serviço associado ao agendamento.
 - status_id: Status atual do agendamento (padrão é 1).
 - data_hora: Data e hora marcadas para o serviço.
 - observacoes: Notas ou observações adicionais sobre o agendamento.
 - criado_em: Data e hora em que o agendamento foi registrado.
- **Tabela log_agendamentos:** Registra o histórico de alterações e mudanças de status dos agendamentos.
 - id: Identificador único do registro de log.
 - agendamento_id: Agendamento que sofreu a alteração.
 - status_anterior: O status em que o agendamento estava antes da mudança.
 - status_novo: O novo status para o qual o agendamento foi alterado.
 - alterado_em: Data e hora em que a alteração ocorreu.
 - alterado_por: Identificação de quem realizou a alteração.

Consultas e Funcionalidades

Descrição das Consultas

1. **Consulta de quantos agendamentos por Cliente:** Exibir todos os agendamentos feitos por um cliente.

```
SELECT
    u.id,
    u.nome_completo,
    COUNT(a.id)
FROM usuarios u
LEFT JOIN agendamentos a ON u.id = a.usuario_id
GROUP BY u.id, u.nome_completo;
```

2. **Consulta sobre quais os serviços mais utilizados por clientes:** Listar os serviços mais utilizados por clientes.

```
SELECT
    s.id,
    s.nome,
    COUNT(a.id) AS total
FROM servicos s
JOIN agendamentos a ON s.id = a.servico_id
GROUP BY s.id, s.nome
ORDER BY total DESC;
```

3. **Consulta de quantos atendimentos tem por serviço:** Gerar um relatório de quantos atendimentos existem por serviço.

```
SELECT
    s.id,
    s.nome,
    COUNT(a.id)
FROM servicos s
LEFT JOIN agendamentos a ON s.id = a.servico_id
GROUP BY s.id, s.nome;
```

4. **Consulta geral sobre quantos atendimentos estão abertos, fechados ou cancelados:** Gerar um relatório sobre quantos atendimentos estão abertos, fechados ou cancelados.

```
SELECT
    sa.nome,
    COUNT(a.id)
FROM status_agendamento sa
LEFT JOIN agendamentos a ON sa.id = a.status_id
GROUP BY sa.id, sa.nome;
```

Descrição das Funcionalidades

1. **Log de cadastro de agendamento:** O sistema insere automaticamente um log, sobre o agendamento cadastrado.

```
CREATE TRIGGER trg_log_agendamento_criado
AFTER INSERT ON agendamentos
FOR EACH ROW
BEGIN
    INSERT INTO log_agendamentos (agendamento_id, status_anterior, status_novo, alterado_por)
    VALUES (NEW.id, NULL, NEW.status_id, 'SISTEMA');
END$$
```

2. **Procedimento de cadastro de usuário:** Uma *procedure* que cadastra um usuário, caso algo dê errado, o cadastro não é realizado.

```
CREATE PROCEDURE sp_cadastrar_usuario(  
    IN p_perfil_id INT,  
    IN p_nome VARCHAR(100),  
    IN p_email VARCHAR(100),  
    IN p_senha VARCHAR(255),  
    IN p_telefone VARCHAR(20)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro';  
    END;  
  
    START TRANSACTION;  
    INSERT INTO usuarios (perfil_id, nome_completo, email, senha, telefone)  
    VALUES (p_perfil_id, p_nome, p_email, p_senha, p_telefone);  
    COMMIT;  
END$$
```

3. **Registro de cancelamento de agendamento:** Quando um agendamento é cancelado, esta trigger faz uma inserção na tabela de log, sobre o agendamento cancelado.

```
CREATE TRIGGER trg_log_agendamento_cancelado  
AFTER UPDATE ON agendamentos  
FOR EACH ROW  
BEGIN  
    IF OLD.status_id <> NEW.status_id THEN  
        INSERT INTO log_agendamentos (agendamento_id, status_anterior, status_novo, alterado_por)  
        VALUES (NEW.id, OLD.status_id, NEW.status_id, 'SISTEMA');  
    END IF;  
END$$
```

4. **Procedimento de cadastro de agendamento:** Uma *procedure* que cadastra um agendamento, o qual não será executado caso algo dê errado.

```
CREATE PROCEDURE sp_cadastrar_agendamento(  
    IN p_usuario_id INT,  
    IN p_servico_id INT,  
    IN p_status_id INT,  
    IN p_data_hora DATETIME,  
    IN p_observacoes VARCHAR(100)  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro';  
    END;  
  
    START TRANSACTION;  
    INSERT INTO agendamentos (usuario_id, servico_id, status_id, data_hora, observacoes)  
    VALUES (p_usuario_id, p_servico_id, p_status_id, p_data_hora, p_observacoes);  
    COMMIT;  
END$$
```
